

Jupyter Notebook 快速入门与常用用法

1.机器学习环境安装

1.1 Miniconda

Miniconda 是 Anaconda 的最小安装版。只包含 conda 和 Python 以及它们依赖的少量软件包，如 pip, zlib。

Miniconda 下载地址：<https://docs.conda.io/en/latest/miniconda.html>

1.2 pip install

下载安装常用软件包，`-i` 参数指定清华镜像下载。

注意：本例安装 tensorflow CPU 版本。如需安装 tensorflow GPU 版本，请替换 tensorflow-cpu 为 tensorflow。

GPU 版 TensorFlow 需安装 CUDA 和 cuDNN，详情见 TensorFlow 中文官网-GPU 支持：<https://tensorflow.google.cn/install/gpu>。

```
1 pip install --upgrade pip
2 pip install -i https://pypi.tuna.tsinghua.edu.cn/simple tensorflow-cpu
```

1.3 Colaboratory[可选]

对于有外网条件的人来说，Colaboratory 可以作为上述配置的替代品。Colaboratory 是 G 开发的在浏览器上运行 Python 的工具，其支持 GPU 和 TPU。使用体验与 Jupyter Notebook 一致，且更方便。以我的使用体验来说，它比个人电脑要快。Colab 支持付费版，性能更高。

2.Jupyter Notebook

打开 Miniconda/Anaconda 命令行，输入启动命令启动 Jupyter。

```
1 jupyter Notebook
```

如下图，进入 Jupyter 启动页，然后点击 New -->Python3 新建 Notebook。现在我们正式进入 Jupyter Notebook 的世界。



2.1 常见用法

打印

键入命令，按 **Ctrl+Enter** 执行单元格。

```
1 print("2020, 戴好口罩!")
```

执行结果如图所示

```
In [1]: print("2020, 戴好口罩!")
2020, 戴好口罩!
```

新建单元格，自动补全

按 **B (Below)** 键，在单元格下方新建单元格。

输入下方代码，按 **Shift+Enter** 执行单元格。试试在输入 **np.linspace** 时，按 **Tab** 键，Jupyter 将自动补全代码。

```
1 import numpy as np
2 myarray = np.linspace(0,10,101)
```

Shift+Enter 将执行选中单元格，并选择下方单元格。如下方无单元格则新建单元格。

```
In [2]: import numpy as np
        myarray = np.linspace(0,10,101)

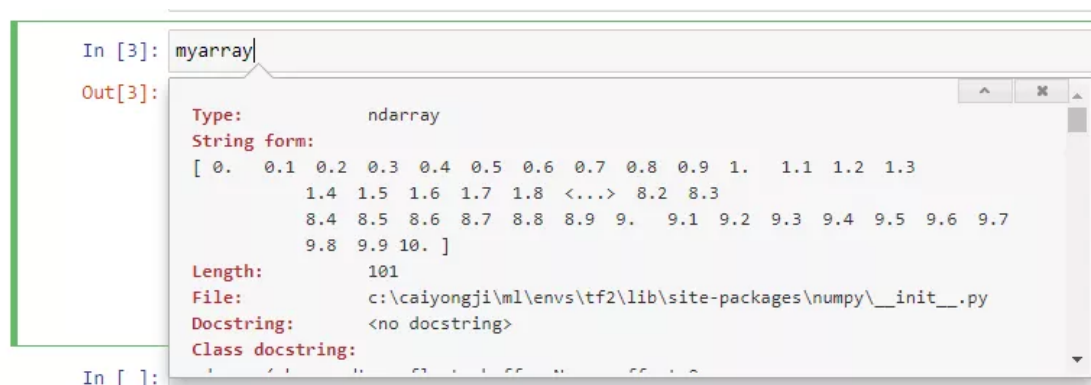
In [ ]: |
```

代码提示

输入 `myarray`，按 **Alt+Enter** 执行选中单元格，并在下方插入新的单元格。我们将光标放在代码后方，按 **Shift+Tab** 查看代码提示。按两次 **Shift+Tab** 可查看代码详情。

```
1 myarray
```

如图显示，`myarray` 是 `ndarray` 类型的数组。



Markdown

选中 `myarray` 单元格，按 **A(Above)** 在 `myarray` 上方插入单元格，再按 **M** 将单元格格式转换为 **Markdown** 格式。

```
1 ## markdown
2 在此处点击**Shift+Tab**可查看myarray数据类型。
```

执行得到如下显示：



序号说明

单元格前面的序号表示单元格执行的次序。其中空 `[]` 为从来没执行过。`[*]` 为尚未执行到。

```
In [24]: from sklearn.metrics import accuracy_score  
y_pred = bag_clf.predict(X_test)  
accuracy_score(y_test, y_pred)
```

Out[24]: 0.912

Feature importance

```
In [*]: from sklearn.datasets import fetch_openml  
  
mnist = fetch_openml('mnist_784', version=1)  
mnist.target = mnist.target.astype(np.uint8)
```

Jupyter 两种模式及全部快捷键

命令模式（按 **ESC** 进入）

Jupyter 常用命令模式快捷键：

快捷键	操作
A	在上方插入单元格
B	在下面插入单元格
D,D	删除选定的单元格
Shift-Enter	运行单元格，在下面选择
Ctrl-Enter	运行选定的单元格
Alt-Enter	运行单元格并在下面插入
Y	将单元格更改为代码格式
M	将单元格更改为Markdown格式

编辑模式（按 **Enter** 进入）

Jupyter 常用编辑模式快捷键：

快捷键	操作
Tab	代码补全或缩进
Shift-Tab	代码提示
Ctrl-D	删除整行
Shift-Enter	运行单元格，在下面选择
Ctrl-Enter	运行选定的单元格
Alt-Enter	运行单元格并在下面插入

Jupyter Notebook 快捷键大全

进入命令模式，按 **H** 查看所有 Jupyter Notebook 快捷键。

Jupyter笔记本电脑有两种不同的键盘输入模式。编辑模式允许您在单元格中键入代码或文本，并以绿色单元格边框指示。命令模式将键盘绑定到笔记本级别的命令，并由带有蓝色左边界的灰色单元格边框指示。
caiyongji偷偷藏入水印。

命令模式 (按 **Esc** 启用)

编辑路径

F : 查找和替换	Shift-J : 在下方扩展所选单元格
Ctrl-Shift-F : 打开命令面板	Ctrl-A : 选择所有单元格
Ctrl-Shift-P : 打开命令面板	A : 在上方插入单元格
Enter : 进入编辑模式	B : 在下面插入单元格
F : 打开命令面板	X : 剪切选定的单元格
Shift-Enter : 运行单元格，在下面选择	C : 复制选定的单元格
Ctrl-Enter : 运行选定的单元格	Shift-V : 在上方粘贴单元格
Alt-Enter : 运行单元格并在下面插入	V : 在下面粘贴单元格
Y : 将单元格更改为代码	Z : 撤消单元格删除
M : 将单元格更改为降价	D , D : 删除选定的单元格
R : 将单元格更改为原始	Shift-M : 合并选定的单元格，或者如果仅选择一个单元格，则将当前单元格与下面的单元格合并
1 : 将单元格更改为标题1	Ctrl-S : 保存和检查点
2 : 将单元格更改为标题2	S : 保存和检查点
3 : 将单元格更改为标题3	L : 切换行号
4 : 将单元格更改为标题4	O : 切换所选单元格的输出
5 : 将单元格更改为标题5	Shift-O : 切换所选单元格的输出滚动
6 : 将单元格更改为标题6	H : 显示键盘快捷键
K : 选择上方的单元格	I , I : 中断内核
Up : 选择上方的单元格	O , O : 重启内核 (带对话框)
Down : 在下面选择单元格	Esc : 关闭传呼机
J : 在下面选择单元格	Q : 关闭传呼机
Shift-K : 在上方扩展所选单元格	Shift-L : 切换所有单元格中的行号，并保留设置
Shift-Up : 在上方扩展所选单元格	Shift-Space : 向上滚动笔记本
Shift-Down : 在下方扩展所选单元格	Space : 向下滚动笔记本

编辑模式 (按 **Enter** 启用)

Tab : 代码完成或缩进	Ctrl-Right : 一个字右转
Shift-Tab : 工具提示	Ctrl-Backspace : 删除单词之前
Ctrl-J : 缩进	Ctrl-Delete : 之后删除单词
Ctrl-L : 下垂	Ctrl-Y : 重做
Ctrl-A : 全选	Alt-U : 重做选择
Ctrl-Z : 撤消	Ctrl-M : 进入命令模式
Ctrl-/ : 评论	Ctrl-Shift-F : 打开命令面板
Ctrl-D : 删除整行	Ctrl-Shift-P : 打开命令面板
Ctrl-U : 撤消选择	Esc : 进入命令模式
Insert : 切换覆盖标志	Shift-Enter : 运行单元格，在下面选择
Ctrl-Home : 转到单元格开始	Ctrl-Enter : 运行选定的单元格
Ctrl-Up : 转到单元格开始	Alt-Enter : 运行单元格并在下面插入
Ctrl-End : 转到单元格末尾	Ctrl-Shift-Minus : 在光标处拆分单元格
Ctrl-Down : 转到单元格末尾	Ctrl-S : 保存和检查点
Ctrl-Left : 剩下一个字	Down : 向下移动光标
	Up : 向上移动光标

关

2.2 进阶用法

LaTeX 数学公式支持

Jupyter Notebook 支持 LaTeX 数学公式。新建 Markdown 代码格式单元格。

输入下方文本将显示 LaTeX 公式，这对我们学习机器学习记数学公式非常有帮助。

```
1 ### variance 方差公式
2  $s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$ 
3 (其中  $\bar{x}$  为均值)
```

variance 方差公式

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}$$

(其中 $\bar{x}$ 为均值)

试试执行下面的代码，看看会显示什么？

```
1 \begin{equation*}
2 \left( \sum_{k=1}^n a_k b_k \right)^2 \leq \left( \sum_{k=1}^n a_k^2 \right) \left( \sum_{k=1}^n b_k^2 \right)
3 \end{equation*}
```

魔法命令

输入 `%lsmagic` 查看所有可用魔法命令。

```
In [8]: %lsmagic

Out[8]: Available line magics:
%alias %alias_magic %await %autocall %automagic %autosave %bookmark %cd %clear %cls %colors %conda %config %connect_info %copy %ddir %debug %dhist %dirs %doctest_mode %echo %ed %edit %env %gui %hist %history %killbgscripts %ldir %less %load %load_ext %loadpy %logoff %logon %logstart %logstate %logstop %ls %lsmagic %macro %magic %matplotlib %mkdir %more %notebook %page %pastebin %pdb %pdef %pdoc %pfile %pinfo %pinfo2 %pip %popd %pprint %precision %prun %psearch %psource %pushd %pwd %pycat %pylab %qtconsole %quickref %recall %rehashx %reload_ext %ren %rep %rerun %reset %reset_selective %rmdir %run %save %sc %set_env %store %sx %system %tb %time %timeit %unalias %unload_ext %who %who_ls %whos %xdel %xmode

Available cell magics:
%%! %%HTML %%SVG %%bash %%capture %%cmd %%debug %%file %%html %%javascript %%js %%latex %%markdown %%perl %%prun %%pypy %%python %%python2 %%python3 %%ruby %%script %%sh %%svg %%sx %%system %%time %%timeit %%writefile

Automagic is ON, % prefix IS NOT needed for line magics.
```

其中比较常见的如 `%pwd`, `%timeit` 等。

```
In [9]: %pwd

Out[9]: 'C:\\caiyingji\\ML\\pythonProject'

In [10]: import numpy as np
%timeit np.sin(99)

1.03 µs ± 17.4 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```